DNS, NGINX Resolver & Reverse Proxy — EKS Notes

1. DNS — The Internet's Address Book

DNS (Domain Name System) converts a hostname into an IP address. Example: **backend.example.com → an IP address**.

> **Memory hook:** DNS is like an address book: “What address belongs to this name?”

2. DNS Resolver

A **DNS resolver** is the DNS server that receives a lookup request and finds the answer. A **DNS resolver IP** is simply the IP address of that DNS server. Examples of public resolvers include 8.8.8.8 and 1.1.1.1.

> **Important:** NGINX's resolver directive tells NGINX which DNS server to ask. It does not mean NGINX itself becomes a DNS server.

3. NGINX Reverse Proxy

A **reverse proxy** sits in front of backend applications. NGINX receives a request from the user and forwards it to the correct backend.

> Typical flow: **User → NGINX → Backend**.

4. NGINX Resolver vs Reverse Proxy

Concept	Simple meaning
NGINX Reverse Proxy	Receives application requests and forwards them to a backend.
NGINX resolver	Tells NGINX which DNS server to use for hostname lookups.
DNS Resolver	The DNS server that answers hostname-to-IP questions.
DNS Resolver IP	The IP address of that DNS server.

5. Your EC2 → EKS Migration Problem

Before migration, the frontend container ran on EC2/Docker and NGINX used a DNS resolver that worked in that environment.

After migration, the frontend ran as a Pod in EKS. Kubernetes introduced its own service-discovery environment, normally using **CoreDNS**. If NGINX was still configured to use an old or unsuitable resolver, it could fail to resolve the backend hostname.

> **Likely failure:** Frontend → NGINX → DNS lookup → wrong/incompatible resolver → backend hostname could not be resolved → backend connection failed.

> **Fix:** The NGINX resolver was changed to a resolver appropriate for the EKS/Kubernetes environment. After that, NGINX could resolve the backend address and the frontend connected successfully.

6. Where NGINX Was Running in Your Case

There was no separate NGINX Pod. NGINX was running **inside the frontend container/Pod**. The resolver configuration was supplied through the frontend deployment/manifest.

> So the architecture was roughly: **User** → **Load Balancer/Frontend Service** → **Frontend Pod** → **NGINX** → **Backend Service** → **Backend Pod(s)**.

7. How Does DNS Know the Backend Pod IP?

In normal Kubernetes communication, you usually do **not** ask DNS for a changing Pod IP directly. Instead, you create a Kubernetes **Service** for the backend.

Kubernetes knows which Pods belong to the Service. CoreDNS provides DNS records for Kubernetes Services. NGINX asks for the Service name, DNS returns the Service address, and Kubernetes networking routes the traffic to an appropriate backend Pod.

> **Example:** NGINX asks for backend-service → CoreDNS returns the Service address → Kubernetes routes traffic → one of the current Backend Pods receives it.

8. Why Use a Service Instead of a Pod IP?

Pod IPs can change when Pods are restarted, rescheduled, or replaced. A Kubernetes Service provides a stable name and service address while the set of backend Pods can change.

> **Good pattern:** NGINX → backend-service → current Backend Pod.
> **Avoid:** NGINX → hard-coded Pod IP.

9. Complete Picture

> User
> ↓
> Frontend Pod
> ↓
> NGINX (reverse proxy)
> ↓
> DNS resolver / CoreDNS (finds the Kubernetes Service)
> ↓
> Backend Service
> ↓
> Kubernetes networking
> ↓
> Backend Pod

10. Key Takeaways

- **DNS:** translates names into addresses.
- **DNS Resolver:** answers DNS lookup questions.
- **NGINX resolver:** tells NGINX which DNS server to ask.
- **NGINX reverse proxy:** forwards HTTP/HTTPS requests to backend applications.
- **CoreDNS:** provides Kubernetes DNS/service discovery.
- **Kubernetes Service:** gives a stable way to reach changing backend Pods.

• **Your migration issue:** NGINX's DNS resolution path changed when the frontend moved from EC2/Docker into EKS; changing the resolver made backend discovery work again.

Memory hook: NGINX is the driver ■, DNS is the address book/GPS ■, the Kubernetes Service is the stable destination name ■, and the Backend Pods are the actual workers inside.